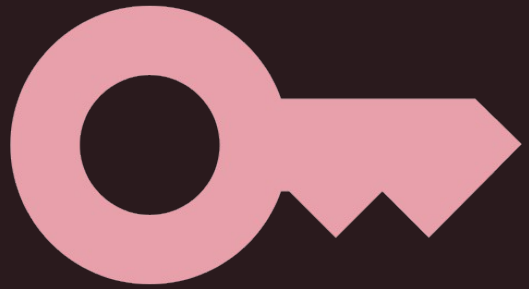
LAB - DAY 2

Higher National Diploma in Network Engineering, Year 2
BSc (Hons) Ethical Hacking & Network Security, Year 2

Ciphers by hand and by machine



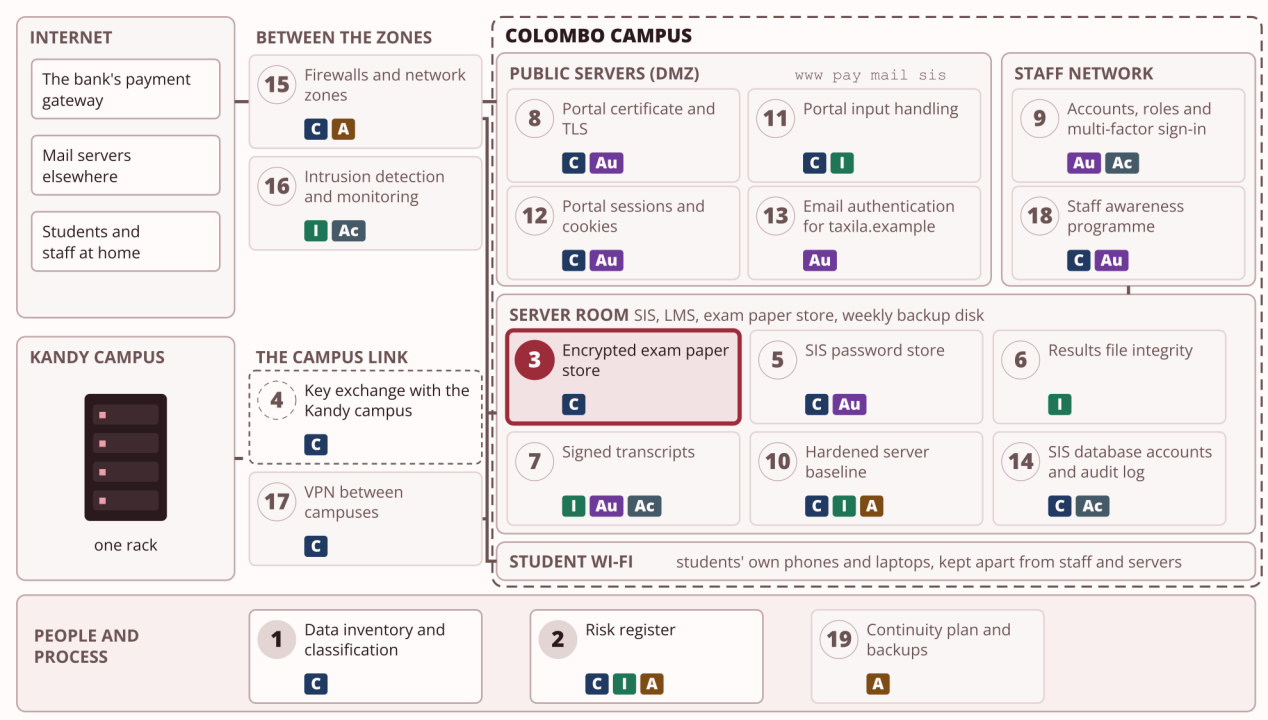
Adds to the register: **3**

Floor tier: every step

Full tier: recommended

Niranga Dharmaratna
School of Computing & Engineering
National Institute of Business Management

The security register



TODAY

3 Encrypted exam paper store

KEY

- Added today
- Added earlier
- Concept only
- Not yet added

R

REGISTRAR

If the folder leaks again, I want whoever takes it to find only ciphertext. Show me that only the key brings the paper back.

The Registrar, at the start of the lab.

Your personal values

```
# plaintext: your index number, capitals, no spaces or hyphens
COHNDNE999X000   (the example belongs to nobody)
# key: SHA-256 of the plaintext, all 64 hex characters
5b02a8f821835e23aa2d08921e40ddb0e0e21220f3e7d4e0a979bc9e962bb99b
# starting value: SHA-256 of the index written backwards, first 32
0def77507f34a59d5dbf643c1c6e82b9
```

A real key is random. This one comes from a public recipe, so it would be a poor real key.

SEGMENT 1 - 25 MIN

Break a shift cipher by hand

Both tiers

Count, guess the shift, confirm

1 Count the letters of `shift-cipher.txt` in `frequency-bars`.

Expected: One letter stands far above the rest; it is probably e.

Floor path: Both tiers do segment 1 in the browser.

2 Turn `caesar-wheel` back by that shift, then try every key.

Expected: A readable message from the examinations office.

Write down the shift and the first sentence. They go in your evidence pack.

SEGMENT 2 - 40 MIN

Encrypt your personal message with the Advanced Encryption Standard

Both tiers

Make your key, then encrypt

3 Run the recipe on the example first; check the values.

Expected: Ciphertext `bc6b159611b059c4db6b8eb5b1ac2add`.

4 Key: the Secure Hash Algorithm (SHA-256) of your index.

Expected: 64 hexadecimal characters.

Floor path: CyberChef `SHA2` with `Size` set to `256`, not the default 512.

Starting value, then encrypt

5

Starting value: SHA-256 of your reversed index, first 32 characters.

Expected: 32 hexadecimal characters.

Floor path: Check your reversal with `Reverse`; then `SHA2` and `Take bytes`, length 32.

6

Encrypt: Advanced Encryption Standard (AES-256), cipher block chaining (CBC).

Expected: 32 hex characters, the same on both tiers.

Floor path: `AES Encrypt`: key and IV as `HEX`, `CBC`, input `Raw`, output `Hex`.

The same on the full tier

```
$ printf '%s' COHNDNE999X000 > plain.txt
$ printf '%s' COHNDNE999X000 | sha256sum | cut -c1-64
$ printf '%s' 000X999ENDNHOC | sha256sum | cut -c1-32
$ openssl enc -aes-256-cbc -nosalt \
    -K <key> -iv <start> -in plain.txt -out cipher.bin
$ xxd -p cipher.bin
bc6b159611b059c4db6b8eb5b1ac2add
```

Use `printf`, not `echo`: `echo` adds a line ending and changes the ciphertext.

SEGMENT 3 - 25 MIN

The wrong key

Both tiers

The right key, then a changed key

7 Decrypt with your key and starting value.

Expected: Exactly your index number.

8 Change the last character of your key and decrypt again.

Expected: It fails. Never your index, never part of it.

Floor path: `AES Decrypt`: input `Hex`, output `Raw`.

```
$ openssl enc -d -aes-256-cbc -nosalt \
    -K <changed key> -iv <start> -in cipher.bin
bad decrypt
```

CyberChef usually shows an error, and sometimes an empty or garbled result.

SEGMENT 4 - 35 MIN

Encrypt to a public key and meet the hybrid idea

Both tiers

Lock with a key you cannot open

9 Encrypt your index to `module-public-key.txt`.

Expected: 512 hex characters; different every run.

Floor path: `RSA Encrypt` with `RSA-OAEP`, then `To Hex`, delimiter `None`.

10 Try to open it yourself.

Expected: You cannot. Only the private key opens it.

11 Encrypt your AES key the same way.

Expected: The hybrid pattern, done by hand.

The same on the full tier

```
$ openssl pkeyutl -encrypt -pubin \
    -inkey module-public-key.txt \
    -pkeyopt rsa_padding_mode:oaep \
    -in plain.txt -out to-module.bin
$ wc -c to-module.bin
256 to-module.bin
$ openssl pkeyutl -decrypt -pubin \
    -inkey module-public-key.txt -in to-module.bin
A private key is needed for this operation
```

Your lecturer holds the private key and opens two volunteers' files in front of the class.

SEGMENT 5 - 25 MIN

Prove it, break it, reset and write up

Both tiers

Prove it, then break it

PROVE IT

- Your key returns **exactly** your index; the changed key does not
- Your ciphertext matches your lecturer's recompute tool

× BREAK IT

- Change the last character of the ciphertext and decrypt
- It fails: one change scrambles a whole 16-byte block

Reset, and the rules

RESET

- Copy your evidence first, then delete the lab files
- Full tier: restore the `day-2-start` snapshot

RULES OF ENGAGEMENT

- No tool in this lab can affect anyone else's system
- You encrypt only your own index and the provided message

Record your evidence

EVIDENCE

- Each value goes in the template's table, with the step it comes from
- The sketch's labels and the 120-word question are in the lab sheet
- The broken shift cipher: the plaintext and the shift you found
- The ciphertext of your personal plaintext and its correct decryption
- The result of decrypting with a changed key
- The hand-drawn two-key sketch
- Your 120-word explanation

Questions & Discussion

Next: Day 3, fingerprints: hashing and integrity.



Disclaimer & Copyright

The information in this presentation is for general teaching purposes only. It is not comprehensive and is not professional advice; any reliance you place on it is at your own risk. I aim to keep it accurate, but I make no representations or warranties of any kind about its completeness, accuracy, reliability or suitability.

Security tools, standards and laws change. Versions, editions and commencement dates in this presentation were checked at the time of writing; check the current documentation before relying on them.

This presentation may refer to third-party websites or content. I do not endorse their views or information, and I am not responsible for their content.

Taxila College is fictional. Its campuses, people, systems and incidents were invented for teaching. Product and tool names, for example Kali Linux, VirtualBox, CyberChef and Wireshark, belong to their owners; this material is not affiliated with or endorsed by any of them.

Practise only on your own virtual machine, the module's provided files and application, and the portal's demos. Testing a system you do not own, without written permission, is an offence under Sri Lanka's Computer Crime Act, No. 24 of 2007.

© Niranga Dharmaratna, School of Computing & Engineering, National Institute of Business Management. Teaching content under CC BY-NC-SA 4.0; code and demonstrations under MIT; Open Sans under the SIL Open Font Licence.

Revision 18d75a8